Method 2: Low-RAM CPU-Capped Cycles Rendering

Physically Based Raytracing with Strict N-Core CPU Locking, AI Denoising & Small-Tile Memory Protection

Tested on: Blender 4.4.3 · Windows · Intel Core i7-1255U (10 cores / 12 threads) · Intel Iris Xe · 16 GB RAM · NVMe SSD. Utilization figures in this guide come from that single machine; yours will differ.
Blender 5.x: the Cycles properties used here were verified on Blender 4.4.3 only; please report any differences.

1. About This Method, Hardware Profile & When It Is Preferred

How the Method Works: Method 2 uses Blender's physically based path-tracer (Cycles) configured specifically for low-RAM and CPU-constrained machines. Normally, Cycles claims 100% of all logical CPU cores, freezes the computer, and consumes massive amounts of RAM building BVH acceleration structures and full-screen buffers. This method locks Blender to an exact number of CPU threads (e.g., 8 threads via `-t 8`), slices each frame into small 256×256 memory tiles, reduces light bounces to the essential minimum, and pairs ultra-low sample counts (4–12 samples) with Intel OpenImageDenoise (Fast Prefilter) to produce clean raytraced frames in seconds.

Hardware Component	Target Utilization	Exact Role in Method 2
CPU (Processor Cores)	65% – 80% (Capped at 8 Threads)	Performs BVH geometry building, ray-triangle intersections, and Intel OpenImageDenoise AI filtering.
GPU (Graphics Card)	0% – 5% (Idle / Free)	Completely free for desktop display or other apps (or an optional GPU toggle for OptiX / oneAPI / HIP).
System RAM (Memory)	Strictly Capped (~1.8 – 3.0 GB)	Protected via 256px auto-tiling, disabled spatial splits, and subdivision capping.
SSD / NVMe Storage	Tile Cache & Fast PNG Output	Caches temporary render tiles if RAM is tight and writes 0%-compressed 8-bit PNG frames.

When Is Method 2 Preferred?

- **Strict CPU Core Quota Required:** When you must restrict Blender to a specific number of CPU cores (such as 8 cores out of 12 or 16) so your PC stays responsive and never overheats.
- **No Dedicated GPU or VRAM Overflow:** When a scene's textures exceed GPU VRAM (causing GPU crashes) or when rendering on a laptop/server with no supported 3D graphics card.
- **Raytraced Realism Needed:** When your scene requires true Cycles glass refraction, mirror reflections, or bounce lighting that EEVEE rasterization cannot approximate.

2. Complete Step-by-Step Guide (From Setup to Final MP4 Video)

Step 1: Unlock OS CPU Turbo Boost

```
# WINDOWS (Run in Command Prompt as Administrator):
powercfg /setactive 8c5e7fda-e8bf-4a96-9a85-a6e23a8c635c

# LINUX (Run in Bash Terminal):
sudo cpupower frequency-set -g performance
```

Step 2: Save the Customizable Python Script (`method2_cycles_low_ram.py`)

Save the script below as `C:\Temp_Render\method2_cycles_low_ram.py` (Windows) or `~/Temp_Render/method2_cycles_low_ram.py` (Linux/macOS). It is also in the repo under `scripts/`.

```
# Method 2 - Low-RAM CPU-Capped Cycles Rendering   [scripts/method2_cycles_low_ram.py]
# Tested on Blender 4.4.3. Run: blender -b --factory-startup SCENE.blend -P THIS_FILE -t 8 -s 1 -e 4440
# -j 1 -a
# --- PART A: USER CUSTOMIZATION SETTINGS ---
import bpy, os, platform

# 1. STRICT CPU CORE / THREAD LIMIT:
# - Set 8 to use exactly 8 CPU threads (e.g., ~70% on a 12-thread i7-1255U).
# - Adjust to 4 on quad-core laptops, or 12-16 on desktop workstations.
CPU_THREADS = 8

# 2. ULTRA-LOW RAM vs. FAST SCENE PERSISTENCE:
# - Set True if your scene uses < 4 GB RAM (saves ~9s reload time per frame!).
# - Set False on 4GB-8GB RAM PCs so Blender frees 100% of RAM after every frame.
KEEP_SCENE_IN_RAM = True

# 3. CYCLES SPEED & QUALITY TUNING:
SAMPLES = 8          # 4-8 for fast animation with AI Denoiser; 32-64 for final stills
MAX_BOUNCES = 2      # 1-2 for fast interior/object renders; 4 for complex glass
TILE_SIZE = 256      # 128 or 256 keeps RAM usage tiny; 2048 for high-RAM PCs
RES_PERCENT = 50     # 50 = 960x540; 75 = 1440x810; 100 = full 1920x1080
FRAME_STEP = 1       # 1 = Render EVERY frame (1, 2, 3...); 2 = Odd frames only

# --- PART B: CYCLES LOW-RAM ENGINE BLOCK (keep directly below Part A) ---
s = bpy.context.scene
r = s.render
c = s.cycles

# Lock Engine & CPU Threads
r.engine = 'CYCLES'
c.device = 'CPU'     # Change to 'GPU' if using NVIDIA OptiX / AMD HIP
s.frame_step = FRAME_STEP
r.threads_mode = 'FIXED'
r.threads = CPU_THREADS
r.use_persistent_data = KEEP_SCENE_IN_RAM
r.use_compositing = False
r.use_sequencer = False
r.use_motion_blur = False

# High-Speed Cycles Sampling & Fast AI Denoising
c.samples = SAMPLES
c.use_adaptive_sampling = True
c.adaptive_threshold = 0.1
c.use_denoising = True
c.denoiser = 'OPENIMAGEDENOISE'      # Runs on CPU via Intel OIDN
c.denoising_prefilter = 'FAST'

# Minimal Light Bounces (Huge CPU Speedup)
c.max_bounces = MAX_BOUNCES
c.diffuse_bounces = 1
c.glossy_bounces = 1
c.transmission_bounces = 1
c.volume_bounces = 0
c.transparent_max_bounces = 2
c.caustics_reflective = False
c.caustics_refractive = False

# Low-RAM Memory Protection (Small Tiles + Subdivision Cap)
c.use_auto_tile = True
c.tile_size = TILE_SIZE
c.debug_use_spatial_splits = False   # Saves hundreds of MB of BVH RAM
r.use_simplify = True
r.simplify_subdivision_render = 1    # Prevents subdivision modifiers from eating RAM
r.simplify_child_particles_render = 0.1

# Resolution & Fast SSD Uncompressed PNG Output
r.resolution_x = 1920
r.resolution_y = 1080
r.resolution_percentage = RES_PERCENT
out = "C:/Temp_Render/" if platform.system() == "Windows" else os.path.expanduser("~/Temp_Render/")
os.makedirs(out, exist_ok=True)
r.filepath = os.path.join(out, "frame_")
r.use_overwrite = False
r.use_placeholder = True
r.image_settings.file_format = 'PNG'
r.image_settings.color_mode = 'RGB'
```

```
r.image_settings.color_depth = '8'
r.image_settings.compression = 0

print(f">>> METHOD 2 (CYCLES {CPU_THREADS}-THREAD LOW-RAM MODE) ACTIVE <<<")
```

Step 3: Run the Headless Render Command (Windows, Linux & macOS)

```
# WINDOWS (Command Prompt):
"C:\Program Files\Blender Foundation\Blender 4.4\blender.exe" -b --factory-startup ^
  "C:\Path\To\Your_Scene.blend" ^
  -P "C:\Temp_Render\method2_cycles_low_ram.py" -t 8 -s 1 -e 4440 -j 1 -a

# LINUX (Bash Terminal):
blender -b --factory-startup "$HOME/Path/To/Your_Scene.blend" \
  -P "$HOME/Temp_Render/method2_cycles_low_ram.py" -t 8 -s 1 -e 4440 -j 1 -a

# macOS (Zsh / Terminal):
/Applications/Blender.app/Contents/MacOS/Blender -b --factory-startup \
  "$HOME/Path/To/Your_Scene.blend" \
  -P "$HOME/Temp_Render/method2_cycles_low_ram.py" -t 8 -s 1 -e 4440 -j 1 -a
```

Step 4: Assemble the Rendered PNG Sequence into an MP4 Video

```
# OPTION A: FFmpeg (Windows / Linux / macOS)
ffmpeg -framerate 60 -i "C:/Temp_Render/frame_%04d.png" ^
  -c:v libx264 -pix_fmt yuv420p -crf 18 "C:/Temp_Render/Final_Cycles_Video.mp4"
```

Option B — Blender only (no FFmpeg installation needed). Save the script below as `make_video_blender.py` (also in the repo under `tools/`), edit `FRAMES_DIR` and `FPS` at the top if needed, then run it. On Windows, use the full path to `blender.exe`.

```
blender -b --factory-startup -P make_video_blender.py
```

```
# Assemble frame_*.png into an MP4 using Blender only (no FFmpeg install needed).
# Run: blender -b --factory-startup -P tools/make_video_blender.py
# Frames are added in sorted filename order, so keep the zero-padded frame_0001.png naming.
import bpy, os, glob, platform, struct

FRAMES_DIR = "C:/Temp_Render/" if platform.system() == "Windows" else
    os.path.expanduser("~/Temp_Render/")
FPS = 60
OUTPUT = os.path.join(FRAMES_DIR, "Final_Video.mp4")

files = sorted(glob.glob(os.path.join(FRAMES_DIR, "frame_*.png")))
if not files:
    raise SystemExit("No frame_*.png files found in " + FRAMES_DIR)

def png_size(path):
    """Read width/height from the PNG header so the video matches the rendered frames."""
    with open(path, "rb") as f:
        head = f.read(24)
    return struct.unpack(">II", head[16:24])

width, height = png_size(files[0])
scene = bpy.context.scene
scene.render.fps = FPS
scene.render.resolution_x = width
scene.render.resolution_y = height
scene.render.resolution_percentage = 100

seq_editor = scene.sequence_editor_create()
# Newer Blender versions rename `sequences` to `strips`; support both.
strips = seq_editor.strips if hasattr(seq_editor, "strips") else seq_editor.sequences
strip = strips.new_image("frames", files[0], 1, 1)
for f in files[1:]:
    strip.elements.append(os.path.basename(f))

scene.frame_start = 1
scene.frame_end = len(files)
scene.render.image_settings.file_format = 'FFMPEG'
scene.render.ffmpeg.format = 'MPEG4'
scene.render.ffmpeg.codec = 'H264'
scene.render.filepath = OUTPUT
bpy.ops.render.render(animation=True)
```

```
print(">>> VIDEO WRITTEN:", OUTPUT, "<<<")
```

The script matches the video size to your rendered frames and works with both the older `sequences` and the newer `strips` sequencer API names.

Step 5: Troubleshooting Common Errors & Instant Fixes

Error Message / Symptom	Root Cause	Exact Fix
Out of memory or sudden process termination	Scene textures or subdivision levels exceed available physical RAM.	Set `KEEP_SCENE_IN_RAM = False`, `TILE_SIZE = 128`, and `r.simplify_subdivision_render = 0`.
Render is using 100% of all CPU cores instead of 8	`-t 8` was omitted or overridden after the script ran.	Keep both `r.threads = 8` in the script AND `-t 8` before `-a` in the CLI command.
Denoised image looks blotchy or blurry	Sample count (4) is too low for dark indoor shadows.	Increase `SAMPLES = 16` and set `c.denoising_prefilter = 'ACCURATE'`.

3. Conclusion & Final Verdict

Final Line: Method 2 (Low-RAM CPU-Capped Cycles) is the most reliable way to render raytraced animations on memory-limited systems while strictly enforcing an 8-core CPU ceiling and preventing Out-of-Memory crashes.